

PROJECT REPORT

HOMEWORK 3

Written and Prepared by:

Roxanne Lutz

University of New Mexico

October 7, 2025

HANDWORKED PROBLEMS

See following pages for the handwritten, worked problems:

Prob 1 | What linear function $ax+b=y$ approximates $f(x)=\sin x$ best in vicinity of $x=\pi/4$? How does this relate to Newton's Method?

Consider the tangent line at $x=\pi/4$ of $f(x)=\sin x$.

$$f'(x) = \cos x$$

$$f'(\pi/4) = \cos(\pi/4) = \sqrt{2}/2 \leftarrow \text{slope of tangent line at } x = \pi/4.$$

$$f(\pi/4) = \sin(\pi/4) = \sqrt{2}/2 \leftarrow \text{point: } (\pi/4, \sqrt{2}/2).$$

$$y = ax + b$$

$$\frac{\sqrt{2}}{2} = \frac{\sqrt{2}}{2} \left(\frac{\pi}{4} \right) + b$$

$$\frac{\sqrt{2}}{2} - \frac{\sqrt{2}}{2} \left(\frac{\pi}{4} \right) = b$$

$$\frac{\sqrt{2}}{2} \left(1 - \frac{\pi}{4} \right) = b \Rightarrow \frac{\sqrt{2}}{2} \left(\frac{4}{4} - \frac{\pi}{4} \right) = \frac{\sqrt{2}}{2} \left(\frac{4-\pi}{4} \right) = \frac{\sqrt{2}(4-\pi)}{8} = b.$$

So, best approximation of linear form $y=ax+b$ of $f(x)=\sin x$ @ $x=\pi/4$ is.

$$y = \frac{\sqrt{2}}{2}x + \frac{\sqrt{2}(4-\pi)}{8}$$

This relates to Newton's Method because Newton's Method relies on tangent lines in order to produce successively closer approximations to $f(x)=0$ for some x . This line (assuming $x_0 = \pi/4$) would be used to produce our next approximation (for whatever x that satisfies:

$0 = \frac{\sqrt{2}}{2}x + \frac{\sqrt{2}(4-\pi)}{8}$). We then repeat the process, finding a new linear approximation with that new x .

Prob 2] Locate the root of $f(x) = e^{-x} - \cos x$ that is nearest $\pi/4$.

In other words: Find x such that $0 = e^{-x} - \cos x$

$$\cos x = \frac{1}{e^x} \Rightarrow \cos x e^x = 1$$

$$x = 0, \text{ or } x = ?$$

by simple
guess and
check.

Much harder to find
algebraically.

* By viewing the graph, there appears to be another root of $f(x)$ that is closer to $\pi/4$ than $x = 0$.

* We will use Newton's method to provide an approximate within a tolerance of 10^{-7} .

Newton's Method: $x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$, $n = 0, 1, 2, \dots$ and x_0 is initial guess.

Let $x_0 = \pi/2$ be our initial guess.

$$\text{Let } f(x) = e^{-x} - \cos x$$

$$\text{Let } f'(x) = -e^{-x} + \sin x.$$

$$\begin{aligned} \textcircled{1} \quad x_1 &= \pi/2 - \frac{e^{-\pi/2} - \cos \pi/2}{-e^{-\pi/2} + \sin \pi/2} = \pi/2 - \frac{1/e^{\pi/2}}{(-\frac{1}{e^{\pi/2}} + 1)} = \pi/2 - \frac{1}{e^{\pi/2}(-\frac{1}{e^{\pi/2}} + 1)} \\ &= \pi/2 - \frac{1}{e^{\pi/2} - 1} \approx 1.3084 = x_1. \end{aligned}$$

$$f(1.3084) = e^{-1.3084} - \cos(1.3084) \approx 0.01086 > \text{tolerance. continue.}$$

$$(2) \quad x_2 = 1.3084 - \frac{e^{-1.3084} - \cos(1.3084)}{-e^{-1.3084} + \sin(1.3084)} = 1.3084 - 0.015609 \approx 1.2928 = x_2$$

$$f(1.2928) = e^{-1.2928} - \cos(1.2928) \approx 0.0000716 > \text{tolerance,}$$

continue.

$$(3) \quad x_3 = 1.2928 - \frac{e^{-1.2928} - \cos(1.2928)}{-e^{-1.2928} + \sin(1.2928)} = 1.2928 - 0.00010427$$

$$= 1.2927 = x_3$$

$$f(1.2927) = e^{-1.2927} - \cos(1.2927) \approx 0.00000294 > \text{tolerance,}$$

continue.

$$(4) \quad x_4 = 1.2927 - \frac{e^{-1.2927} - \cos(1.2927)}{-e^{-1.2927} + \sin(1.2927)} = 1.2927 - 4.28 \times 10^{-6}$$

$$f(1.2926957) = e^{-1.2926957} - \cos(1.2926957) \approx -0.0000000133 < \text{tolerance}$$

$\approx 1.2926957 = x_4$
 adding precision to $\pi/4$ sh.
 close enough within
 our tolerance.
STOP.

We will say then that the other root near $\pi/4$ is about $x = 1.2927$ (acknowledging that some precision loss will provide a less-tight approximation).

Now, which is closer? $x=0$ or $x=1.2927$?

$$|\pi/4 - 0| = \pi/4 \approx 0.78540.$$

$$|\pi/4 - 1.2927| = 0.50730 \rightarrow \text{this is the closer root.}$$

Therefore, the r such that $f(r) = 0$ closest to $x = \pi/4$ is approximately $x = 1.2927$.

Prob 3 } For what values will Newton's Method converge for solving $f(x)=0$: $f(x) = \frac{x^2}{1+x^2}$

Consider Newton's Method for this function.

$$f'(x) = \frac{(1+x^2)(2x) - x^2(2x)}{(1+x^2)^2} = \frac{2x + 2x^3 - 2x^3}{(1+x^2)^2} = \frac{2x}{(1+x^2)^2}$$

Newton's Method:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}, n=0, 1, 2, \dots$$

$$\text{So: } x_{n+1} = x_n - \frac{\frac{x_n^2}{1+x_n^2}}{\frac{2x_n}{(1+x_n^2)^2}} = x_n - \frac{x_n^2}{1+x_n^2} \cdot \frac{(1+x_n^2)^2}{2x_n} = x_n - x_n \left(\frac{1+x_n^2}{2} \right)$$

$$x_{n+1} = x_n \left(1 - \frac{1+x_n^2}{2} \right)$$

After viewing the graph, had idea that there is likely some oscillation at a certain $\pm x=a$ in which values $(-a, a)$ converge to root $x=0$, and values outside diverge. due to symmetry.

We can solve mathematically for this a by noting convergence will occur when

$$|x_{n+1}| < |x_n|$$

$$|x_n \left(1 - \frac{1+x_n^2}{2} \right)| < |x_n| \rightarrow -3 < -x_n^2 < 1$$

$$|1 - \frac{1+x_n^2}{2}| < \left| \frac{x_n}{x_n} \right| = 1$$

$$-1 < 1 - \frac{1+x_n^2}{2} < 1$$

$$-1 < \frac{2 - 1 - x_n^2}{2} < 1$$

$$-1 < \frac{1 - x_n^2}{2} < 1$$

$$-2 < 1 - x_n^2 < 2$$

$$3 > x_n^2 > -1$$

$$-1 < x_n^2 < 3$$

$$-1 < x_n^2 \rightarrow \text{True for all such } x_n, \text{ no boundary here to find}$$

$$\sqrt{x_n^2} < \sqrt{3}$$

$$- \sqrt{3} < x_n < \sqrt{3}$$

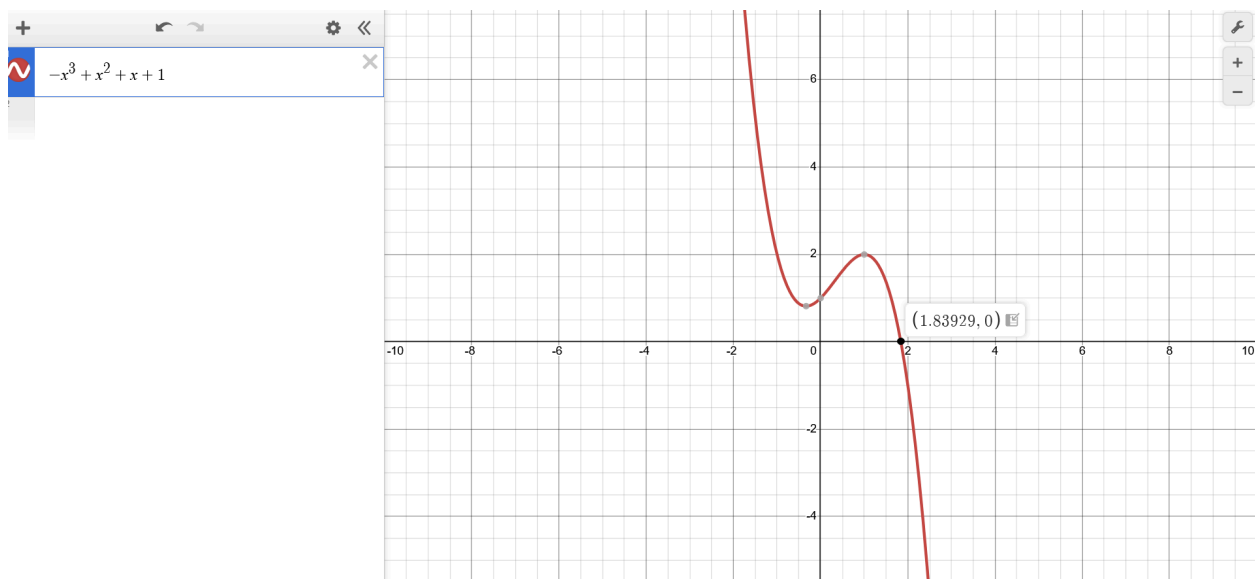
Therefore, Newton's Method converges for all values $x_n \in (-\sqrt{3}, \sqrt{3})$

COMPUTER/MATLAB PART 1

ORIGINAL PROBLEM STATEMENT

“Use Newton’s method to approximate, to within 10^{-8} , the root of the equation $x^3 = x^2 + x + 1$. Hint: You are allowed to guide your initial guess, x_0 , by looking at the graph of $f(x)$.”

First, we can slightly rearrange the function to be $f(x) = -x^3 + x^2 + x + 1$. Following the hint by graphing the function on Desmos, we can see our root should be right around $x = 1.83929$.



So, we will set our initial guess to $x_{init} = 2$ for Newton’s method. We will also need $f'(x)$ which is found as follows by hand (very simply in this case) and then coded as a function to calculate as seen in the pasted `function_01.m` code.

Finding $f'(x)$ by hand:

$$f(x) = -x^3 + x^2 + x + 1.$$

$$f'(x) = -3x^2 + 2x + 1.$$

And now, pasted below is (1) the code for the Newton's method function, which has been rewritten as a general use function based off of implementation from homework 2 code, (2) $f(x) = -x^3 + x^2 + x + 1$ and $f'(x) = -3x^2 + 2x + 1$ in a single function for ease of use, and (3) the main script to feed the function to Newton's method with initial guess, tolerance, and then plot.

```
%
=====
(1) NEWTON'S METHOD WRITTEN AS FUNCTION IN newtonsMethod.m FILE
%
=====

function [x_approx, y_approx, iterations_needed, success] =
newtonsMethod(tolerance, iterations, x_start, f)

x_curr = x_start; % initial guess
% flag for a successful approximation within these parameters
success = false;
% for counting the actual iterations used for easy printing
iterationsNeeded = 0;

for i = 1:iterations;
    [y, dy] = function1(x_curr); % get f(x) and f'(x)
    % hold values for plotting
    xapprox(i) = x_curr;
    yapprox(i) = y;
    if abs(y) < tolerance; % found 0 within tolerance
        success = true;
        break;
    end
end
```

```

    end;
    iterationsNeeded = iterationsNeeded + 1; % for printing only
    x_curr = x_curr - (y ./ dy); % apply newtons method
end;

end % end function

%
=====
(2)  $f(x) = -x^3 + x^2 + x + 1$  AND  $f'(x)$  WRITTEN AS FUNCTION IN
function_01.m FILE
%
=====

function [y, dy] = function_01(x)

    y = ((-1) * (x.^3)) + (x.^2) + x + 1; % f(x)
    dy = ((-3) * (x.^2)) + (2 * x) + 1; % f'(x)

end % end function

%
=====
(3) APPROXIMATING ROOT IN script_01.m FILE
%
=====

tolerance = 10 ^ (-8); % tolerance specified by problem
iterations = 20; % chosen arbitrarily to get approximation
x_init = 2; % initial guess based on graph

% run newtons method with given method
[x_approx, y_approx, iterations_needed, success] = ...
    newtonsMethod(tolerance, iterations, x_init, @(x)
function_01(x));

% print success if we've found a zero within iterations and
tolerance
if success;
    disp( ...
        "Approximate root found successfully in " ...

```



```

        + iterations_needed ...
        + " iterations: x = " ...
        + x_approx(end) ...
    );
else;
    disp( ...
        "Approximate root was NOT found within tolerance in " ...
        + iterations + " iterations." ...
    );
end;

% get exact values of function to plot against approximations
h = 0.001; start_pt = -1; end_pt = 2; % initialization values
x_exact = [start_pt:h:end_pt]; % calculate exact for these vals
of x
y_exact = zeros(size(x_exact)); % preallocate memory

counter = 1;
for x = x_exact;
    [y, dy] = function_01(x);
    y_exact(counter) = y;
    counter = counter + 1;
end;

% plot the exact graph versus the approximation points
figure(1);
plot(x_exact, y_exact, "--c", ...
    x_approx, y_approx, "oy", ...
    x_approx(1), y_approx(1), "og", ...
    x_approx(end), y_approx(end), "or", ...
    LineWidth=1.5);
hold on;
legend( ...
    'Exact Values of f(x)', ...
    "Approximations to the Root with Newton's Method", ...
    "Initial Guess, x = " + x_init, ...
    "Final Approximation" ...
);
title("Exact f(x) vs Approximations to the Root using Newton's
Method");

```

```

xlabel("Domain, [" + start_pt + ", " + end_pt + "] with h = " +
h);
ylabel('Range, output of f(x) and Approximations');
grid on;

%
=====

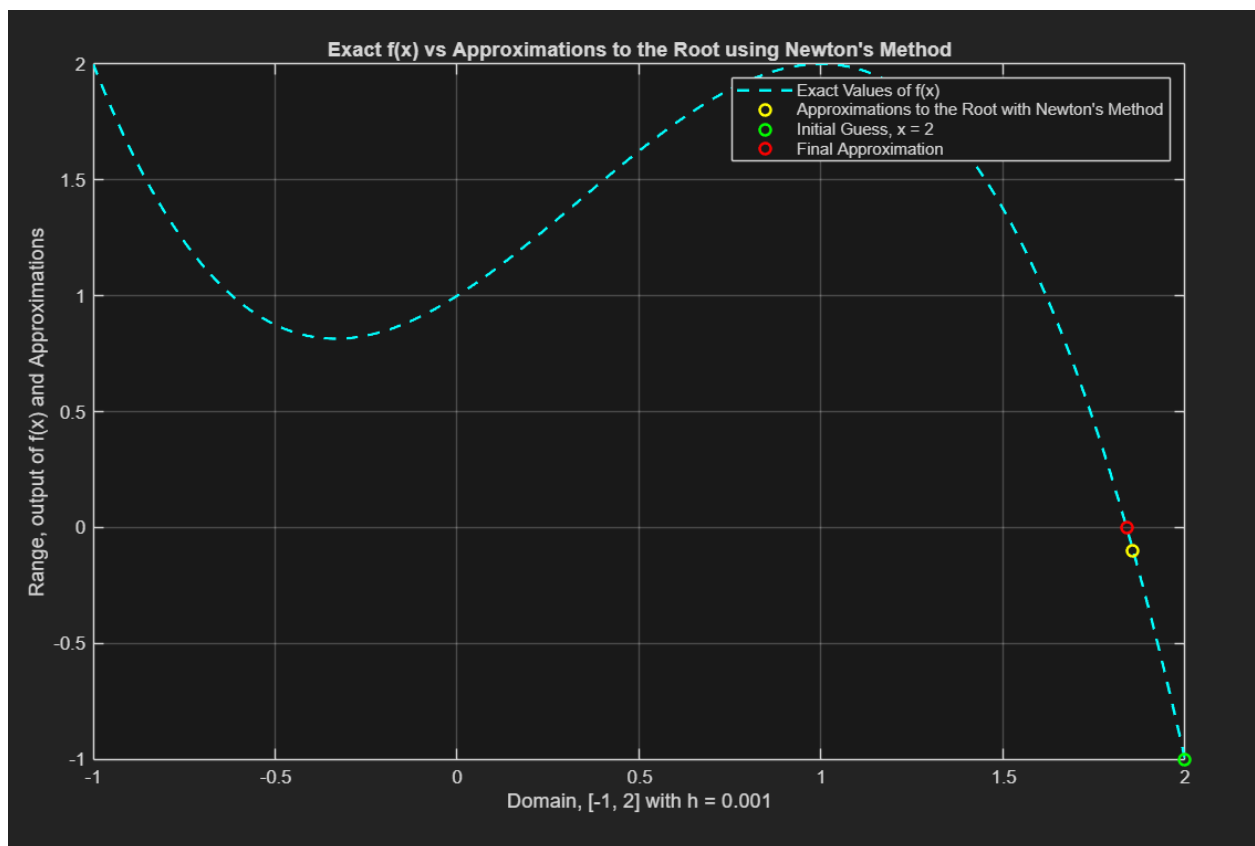
    END CODE FOR COMP PROB 1
%
=====

```

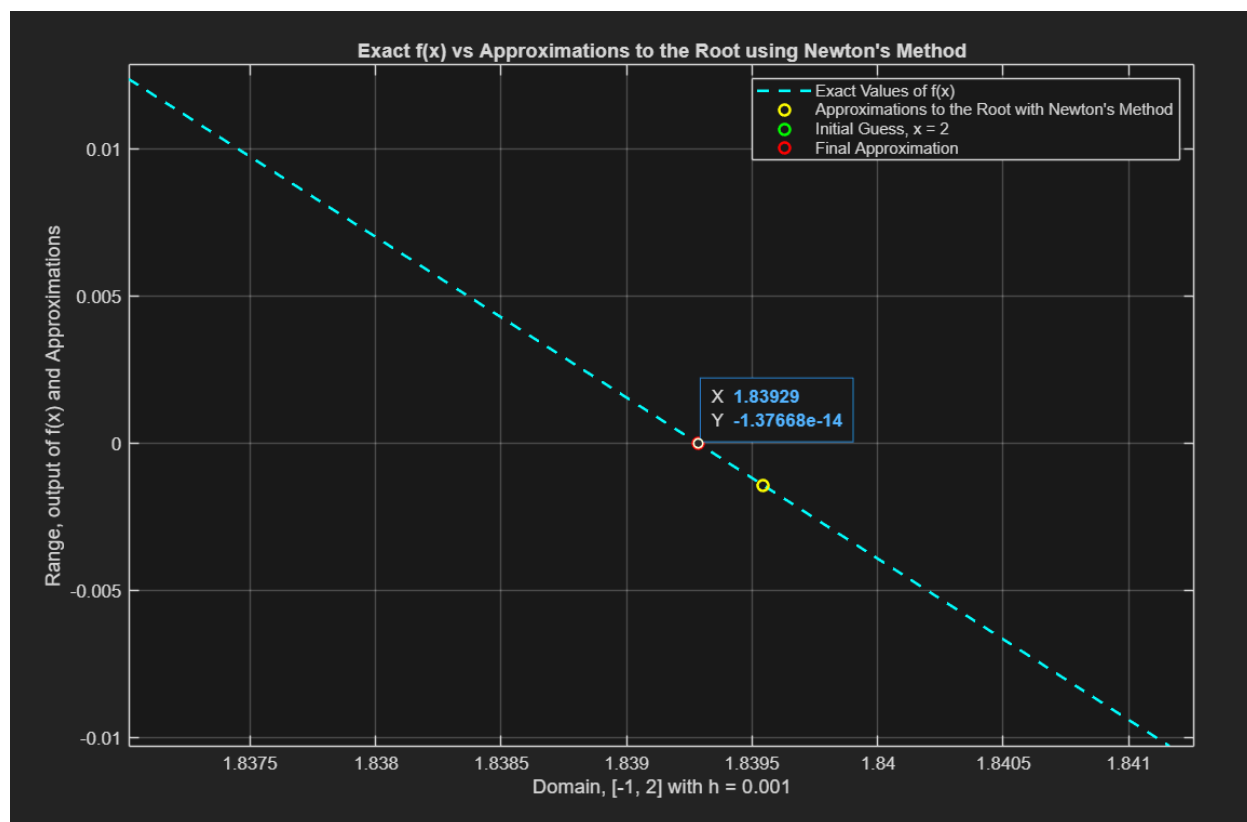
Running the above script ends up taking 4 iterations with $x_{init} = 2$ to find the approximate root $x = 1.8393$, which agrees with our original plotting.

Approximate root found successfully in 4 iterations: $x = 1.8393$

Below is a graphical plot to show the exact function values versus each successive approximation in those 4 iterations, including the initial guess in green.



We can zoom in to see the finer approximations on a much smaller scale. The final two approximations can be seen closer below.



So, we get a final, closer approximation of $x = 1.83929$, which exactly matches what we expect to a 10^{-8} tolerance.

One limitation to note with this function is to consider other values of x_{init} . If we choose $x_{init} = 1$, we will never find the root. And why? Because: $f'(1) = -3(1^2) + 2(1) + 1 = -3 + 2 + 1 = 0$. Obviously, this causes problems with Newton's method because it relies on the presumption that $f'(x) \neq 0$, and this x_{init} guess violates the age-old law of not dividing by 0. Therefore, even if $x = 1$ appears a reasonable guess, it showcases the importance of having a decent original guess, since a poor one can break the approximations quickly.

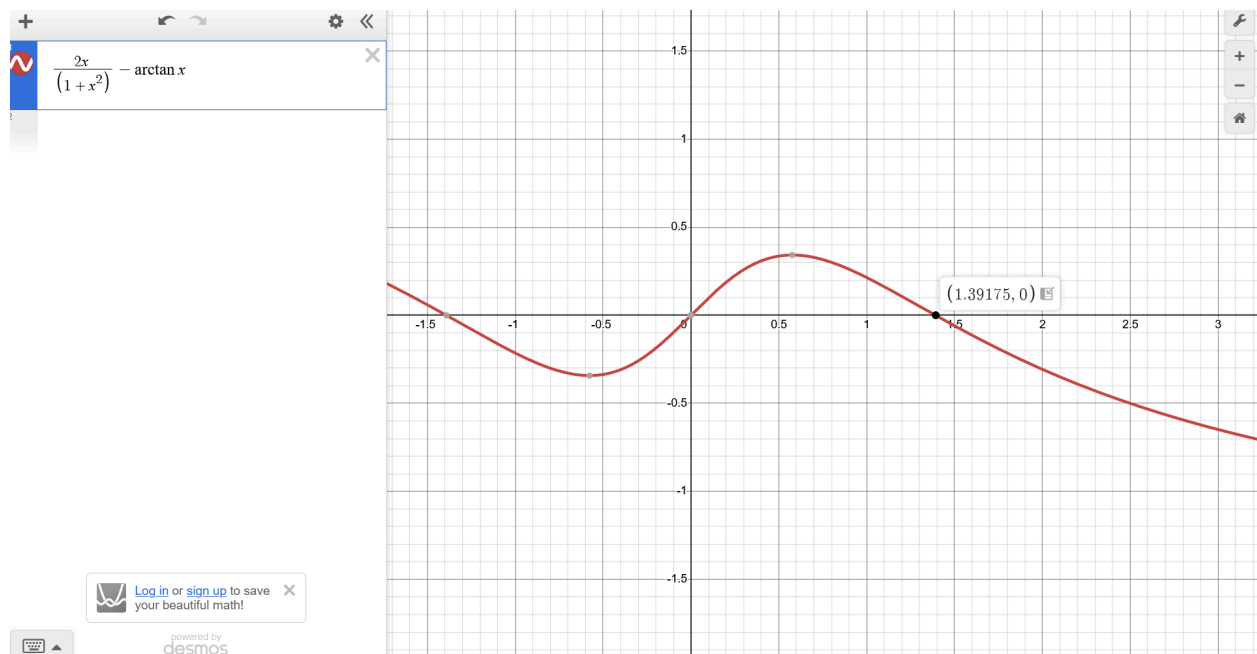
COMPUTER/MATLAB PART 2

ORIGINAL PROBLEM STATEMENT

“Using the bisection method, find the positive root of $2x(1 + x^2)^{-1} = \arctan x$.”

In order to make the problem solvable as a $f(r) = 0$ root finding problem, we will represent the function slightly algebraically rearranged: $f(x) = 2x(1 + x^2)^{-1} - \arctan x$, or even further, $f(x) = 2x / (1 + x^2) - \arctan x$. We luckily do not need to find the derivative of this function due to the Bisection Method not needing a derivative value for x , as opposed to Newton’s Method.

For this problem, we will again utilize Desmos as a secondary graphing tool to guide our initial interval selection for the Bisection Method and confirmation of root value. As we can see, the positive root should be somewhere around $x = 1.39175$.



So, we will employ the Bisection Method to approximate this root by setting our initial interval starting at $x_{start} = 1$ and $x_{end} = 2$. We will also require a tolerance of 10^{-8} as we did for Computer Problem 1 simply for consistency.

Pasted below is the code to accomplish (1) implementing the Bisection Method for finding a root as a function, (2) coding $f(x) = 2x / (1 + x^2) - \arctan x$ as a method to be utilized, and (3) the script to use the Bisection Method to find the root of our function, followed by plotting the exact values versus the midpoint approximations.

⌘

=====

(1)BISECTION METHOD WRITTEN AS FUNCTION IN bisectionMethod.m
FILE

```

%
=====

function [x_mids, y_mids, iterations_needed, success] =
bisectionMethod(tolerance, iterations, x_start, x_end, f)

y_start = f(x_start); % initial evaluation at x1
success = false; % init success flag
iterations_needed = 0; % for easy printing

for i = 1:iterations;

    x_mid = (x_end + x_start) ./ 2; % find mid x val between x1
and x2
    y_mid = f(x_mid); % evaluate at mid point
    % save mid values for return plotting
    x_mids(i) = x_mid;
    y_mids(i) = y_mid;

    % if we've gotten close enough to a root, stop, success!
    if (abs(y_mid) < tolerance);
        success = true;
        break;
    end;

    if (y_start * y_mid > 0); % if both pos or neg y vals
        % set mid value as new lower bound on interval
        x_start = x_mid;
        y_start = y_mid;
    else;
        % otherwise, set mid value as the new upper bound on
interval
        x_end = x_mid;
    end;

    iterations_needed = iterations_needed + 1;
end;

end % end function

```

```

%
=====
(2)  $f(x) = 2x / (1 + x^2) - \arctan x$  WRITTEN AS FUNCTION IN
function_02.m FILE
%
=====

function [y] = function_02(x)
    y = ((2 .* x) ./ (1 + (x .^ 2))) - atan(x);
end % end function

%
=====
(3) APPROXIMATING ROOT IN script_02.m FILE
%
=====

tolerance = 10 ^ (-8); % tolerance specified by problem
iterations = 30; % chosen arbitrarily to get approximation
x_start = 1; % lower bound on interval to find root in
x_end = 2; % upper bound on interval to find root in

[x_mids, y_mids, iterations_needed, success] =
bisectionMethod(tolerance, iterations, x_start, x_end, @(x)
function_02(x));

% print success if we've found a zero within iterations and
tolerance
if success;
    disp( ...
        "BISECTION: Approximate root found successfully in " ...
        + iterations_needed ...
        + " iterations: x = " ...
        + x_mids(end) ...
        );
    % disp("root found: x = " + x_mids(end));
else;
    disp( ...
        "BISECTION: Approximate root was NOT found within
tolerance in " ...
        + iterations + " iterations." ...
    )
end

```

```

    );
end;

% get exact values of function to find root of
h = 0.001; start_pt = -3; end_pt = 3; % initialization values
x_exact = [start_pt:h:end_pt]; % calculate exact for these vals
of x
y_exact = zeros(size(x_exact)); % preallocate memory
counter = 1;

for x = x_exact;
    y_exact(counter) = function_02(x);
    counter = counter + 1;
end;

% plot the exact graph versus the approximation points
figure(1);
plot(x_exact, y_exact, "--c", ...
     x_mids, y_mids, "oy", ...
     x_start, function_02(x_start), "og", ...
     x_end, function_02(x_end), "ob", ...
     x_mids(end), y_mids(end), "or", ...
     LineWidth=1.5);
hold on;
legend( ...
    'Exact Values of f(x)', ...
    '(x, y) Approx Midpoints', ...
    'Initial Starting Interval Value', ...
    'Initial Ending Interval Value', ...
    'Final Approximation'...
);
title('Derived f(x) vs Approximations to the Root using
Bisection Method');
xlabel("Domain, [" + start_pt + ", " + end_pt + "] with h = " +
h);
ylabel('Range, output of f(x) and Approximations');
grid on;

%
=====
END CODE FOR COMP PROB 2

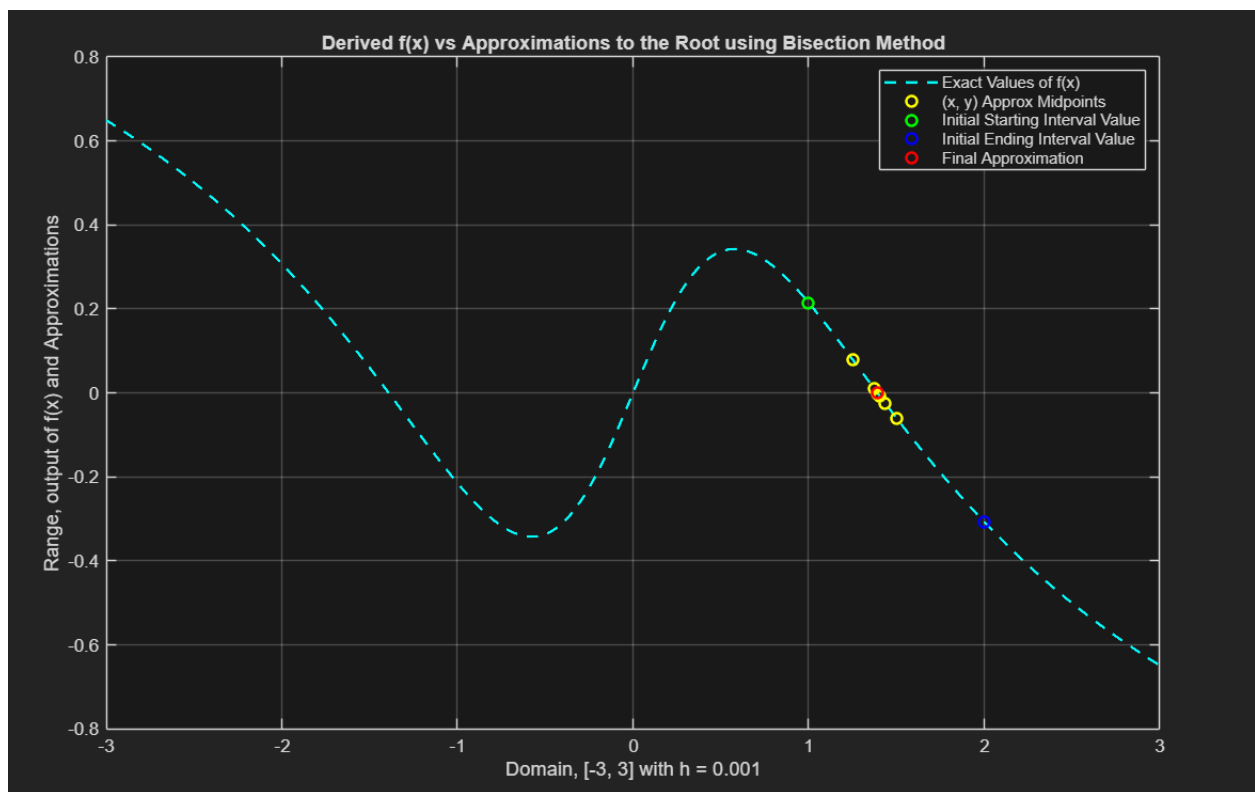
```


⌘

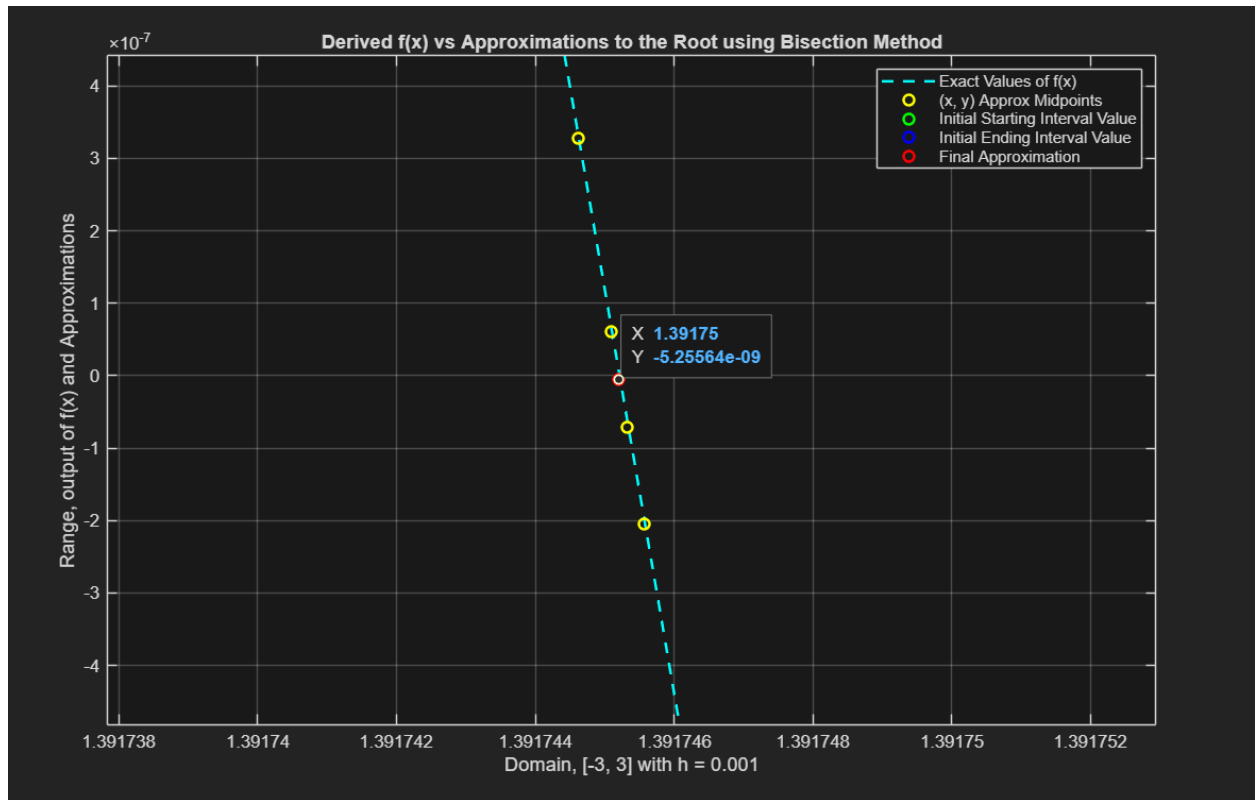
Running the code above takes 22 iterations with the beginning interval endpoints given as $x_{start} = 1$ and $x_{end} = 2$ to come within a tolerance of 10^{-8} of $f(r) = 0$.

BISECTION: Approximate root found successfully in 22 iterations: $x = 1.3917$

As we can see below, this is somewhat of a brute force approach, but still does well at coming to the approximation of our root.



We expect a root close to $x = 1.39175$, and so we will likewise zoom in on the cluster of closer mid-point approximations to see how close that aligns with our expectation and exact function.



As we can see, the approximation we receive from this method is very tightly within our tolerance.

However, to just explore the difference between the two methods, let's find the same root with Newton's Method. We first have to go the extra step of hand deriving the derivative.

$$\begin{aligned}
 f(x) &= 2x(1+x^2)^{-1} - \arctan x \\
 f'(x) &= 2(1+x^2)^{-1} + 2x(-1)(1+x^2)^{-2}(2x) - (1+x^2)^{-1} \\
 &= 2(1+x^2)^{-1} - 4x^2(1+x^2)^{-2} \\
 &= \frac{2}{(1+x^2)} - \frac{4x^2}{(1+x^2)^2} \\
 &= \frac{2(1+x^2) - 4x^2}{(1+x^2)^2} \\
 &= \frac{2 + 2x^2 - 4x^2}{(1+x^2)^2} \\
 &= \frac{2 - 2x^2}{(1+x^2)^2}
 \end{aligned}$$

Then, we will add it to our function.

```
%
=====
(2)  $f(x) = 2x / (1 + x^2) - \arctan x$  and  $f'(x)$  WRITTEN AS
FUNCTION IN function_02.m FILE
%
=====

function [y, dy] = function_02(x)
    y = ((2 .* x) ./ (1 + (x .^ 2))) - atan(x);
    dy = ((-3 .* (x .^ 2)) + 1) ./ ((1 + (x .^ 2)) .^ 2);
end

%
=====
END CODE FOR COMP PROB 2
%
=====
```

And finally, we will simply run it through Newton's method as we did for Computer Problem 1. We will choose an initial guess as $x_{init} = 1$.

```
%
=====
EXTRA CODE IN script_02.m FILE, LINES 64 - END
%
=====

x_init = 1; % init value close to root

% run newtons method with given method
[x_approx, y_approx, iterations_needed, success] = ...
    newtonsMethod(tolerance, iterations, x_init, @(x)
function_02(x));

% print success if we've found a zero within iterations and
tolerance
if success;
    disp( ...
```

```

        "NEWTON: Approximate root found successfully in " ...
        + iterations_needed ...
        + " iterations: x = " ...
        + x_approx(end) ...
    );
else;
    disp( ...
        "NEWTON: Approximate root was NOT found within tolerance
in " ...
        + iterations + " iterations." ...
    );
end;

% plot the exact graph versus the approximation points
figure(2);
plot(x_exact, y_exact, "--c", ...
    x_approx, y_approx, "oy", ...
    x_approx(1), y_approx(1), "og", ...
    x_approx(end), y_approx(end), "or", ...
    LineWidth=1.5);
hold on;
legend( ...
    'Exact Values of f(x)', ...
    "Approximations to the Root with Newton's Method", ...
    "Initial Guess, x = " + x_init, ...
    "Final Approximation" ...
);
title("Exact f(x) vs Approximations to the Root using Newton's
Method");
xlabel("Domain, [" + start_pt + ", " + end_pt + "] with h = " +
h);
ylabel('Range, output of f(x) and Approximations');
grid on;

%
=====

    END CODE FOR COMP PROB 2

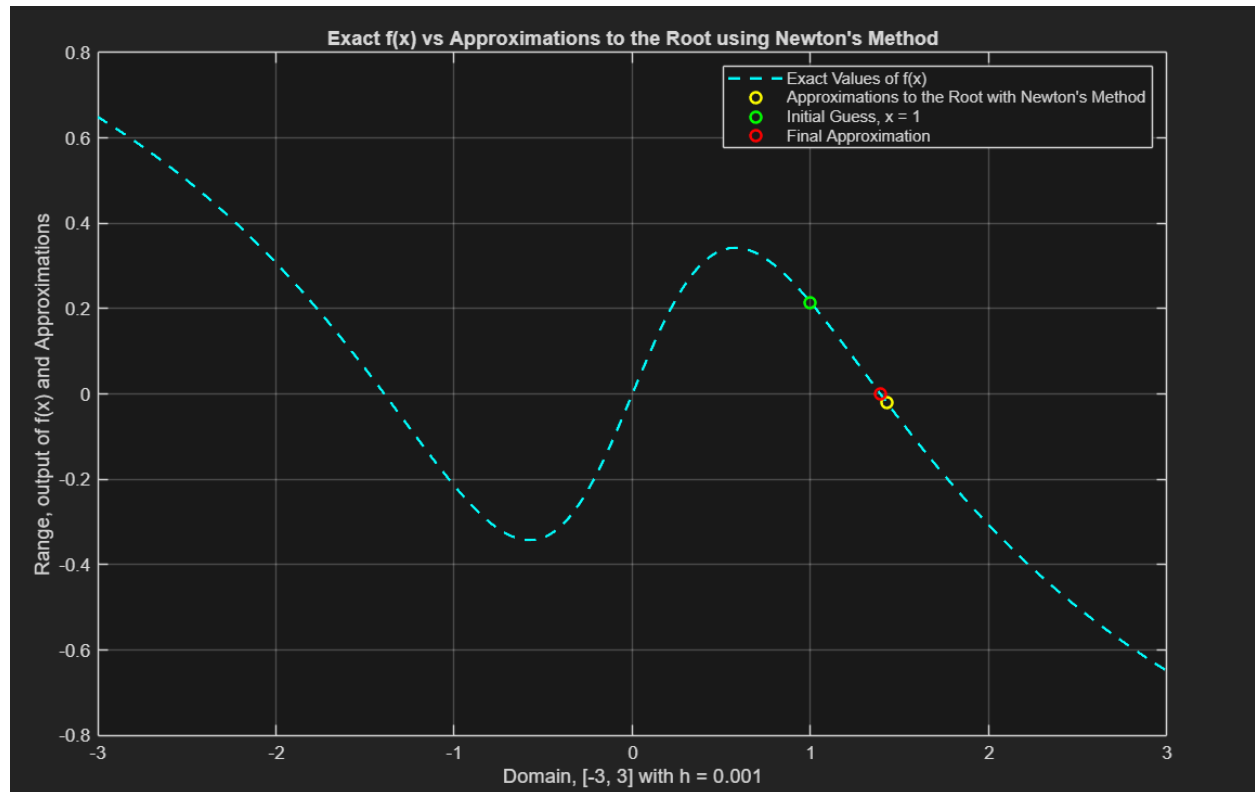
%
=====

```

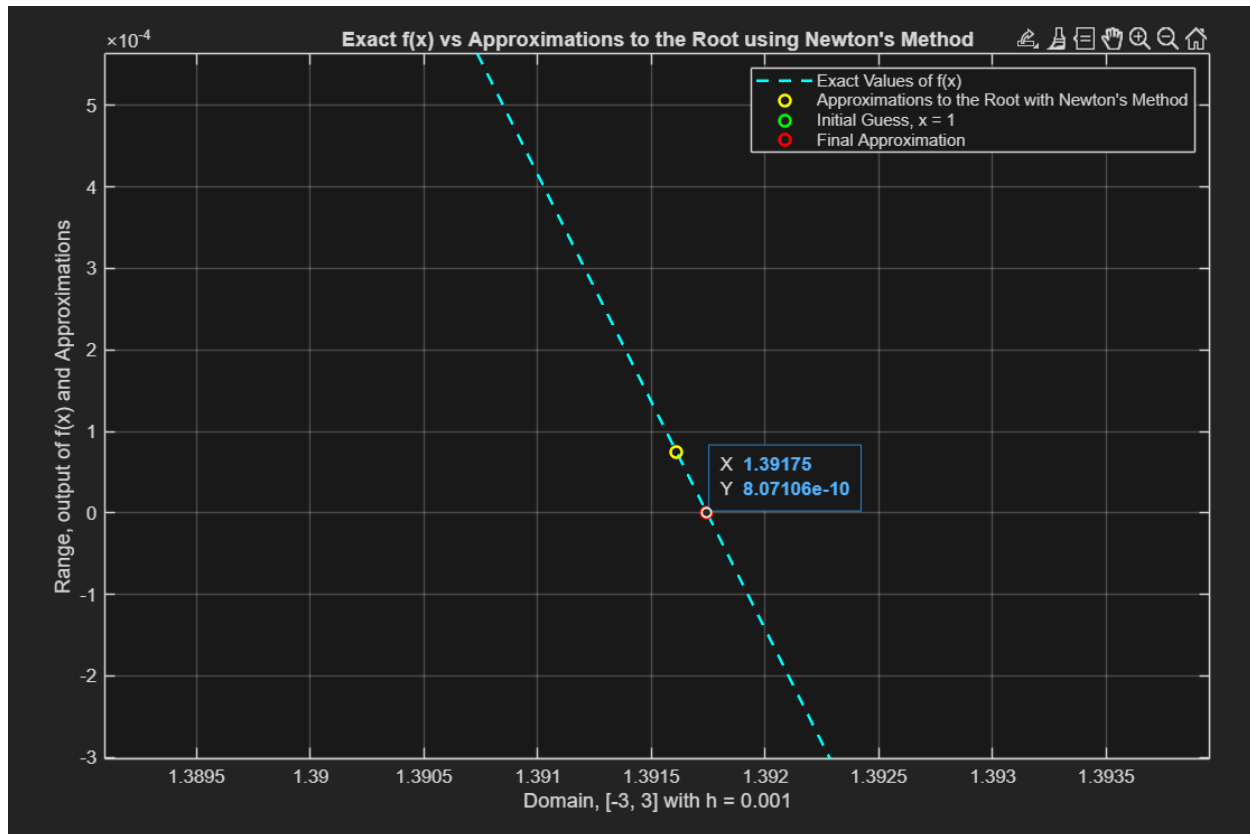
We find that Newton's Method takes 3 iterations only to find this root $x = 1.39175$, as seen below.

```
BISECTION: Approximate root found successfully in 22 iterations: x = 1.3917
NEWTON: Approximate root found successfully in 3 iterations: x = 1.3917
```

And graphically:



We can once again zoom in on the closer approximations to verify our findings.



And interesting note is that the Bisection Method's final y approximation within 10^{-8} to the root was $y = -5.25564 * (10^{-9})$. Here we see a closer approximation of $y = 8.07106 * (10^{-10})$. So, not only does Newton's provide a tighter approximation, but it does it in about $1/7$ of the approximations that Bisection requires.

A tradeoff to consider, however, is that Bisection does not require finding the derivative of a function. The given $f(x)$ was a relatively simple derivative, but still required some careful handwork and algebra to find, code, and ensure correctness of. This could take far more pain with more complex functions. This payoff for Newton's was surely the need for fewer iterations to approximate, but it could also add to the computation by adding an extra dy to calculate with every iteration, which is still worth noting.

We also labor under the assumption that we will not hit a point in the process in which $f'(x) = 0$, which is a risk with Newton's—although, in fairness, an undefined $f(x)$ is a risk with Bisection, so continuity must be considered within an interval chosen.

Further, it is possible that the initial guess to the root will be completely faulty, failing to converge entirely, as we saw for certain values in Handworked Problem 3. This is less of an issue

for Bisection, since as long as a root is within the interval, it should find it with enough iterations.

However, 7 times the iterations needed to find the same root is a notable jump when using Bisection, so depending on the situation, it could very well be worth utilizing Newton's instead.

COMPUTER/MATLAB PART 3

ORIGINAL PROBLEM STATEMENT

“Solve the system of non-linear equations by first eliminating y and then solving the resulting equation in x by Newton's method. Initiate your Newton's method with $x_0 = 1.0$. The system is given by

$$\begin{aligned}x^3 - 2xy + y^7 - 4(x^3)y &= 5 \\ y \sin x + 3(x^2)y + \tan x &= 4.”\end{aligned}$$

We will begin by isolating y in the second equation, and then we will substitute that y into the first. After some algebraic rearranging, we will also find $f'(x)$ in order to use Newton's Method.

Comp Prob 3 | (Handwritten work to show).

Goal: eliminate y , then solve result w/ Newton's method, $x_0 = 1.0$.

$$\begin{cases} x^3 - 2xy + y^7 + 4x^3y = 5 \\ y \sin x + 3x^2y + \tan x = 4 \end{cases}$$

→ Choose equation #2 to rearrange since it's easier to pull out y .

$$y \sin x + 3x^2y + \tan x = 4$$

$$y(\sin x + 3x^2) = 4 - \tan x$$

$$y = \frac{4 - \tan x}{\sin x + 3x^2}$$

→ plug into equation #1 and rearrange to a root finding problem:

$$f(x) = x^3 - 2x\left(\frac{4 - \tan x}{\sin x + 3x^2}\right) + \left(\frac{4 - \tan x}{\sin x + 3x^2}\right)^7 + 4x^3\left(\frac{4 - \tan x}{\sin x + 3x^2}\right) - 5 = 0$$

We also need $f'(x)$: / ~~Let's take this further...~~

$$f(x) = x^3 - 2xy + y^7 + 4x^3y - 5$$

$$f'(x) = 3x^2 - 2y - 2xy' + 7y^6y' + 12x^2y + 4x^3y'$$

$$\text{and } y' = \frac{(\sin x + 3x^2)(-\sec^2 x) - (4 - \tan x)(\cos x + 6x)}{(\sin x + 3x^2)^2}$$

From there we can code the (1) function and its derivative and (2) create a main script to plot the function first on the maximum and minimum values of the approximation vectors (so that the graph is more visualizable), and then plot the function on a smaller domain and range to get a more focused view. We will reuse the same Newton's Method code as created in Computer Problem 1. We will feed Newton's Method an initial guess of $x_{init} = 1$ as specified and keep a tolerance of 10^{-8} .

```
%
=====
(1)  $f(x)$  and  $f'(x)$  WRITTEN AS FUNCTION IN function_03.m FILE AS
SEEN IN HAND DERIVATION
%
=====

function [y, dy] = function_03(x)
    % this is the y substitution value for clear reading
    y_sub = (4 - tan(x)) ./ (sin(x) + (3 .* (x ^ 2)));

    % helpers for coding dy_sub
    lowdhigh = (sin(x) + 3 * (x .^ 2)) .* (-(sec(x) .^ 2));
    highdlow = (4 - tan(x)) .* (cos(x) + (6 .* x));
    lowlow = (sin(x) + 3 .* (x .^ 2)) .^ 2;

    dy_sub = (lowdhigh - highdlow) ./ lowlow;

    % now find y and dy
    y = (x .^ 3) - (2 .* x .* y_sub) + (y_sub .^ 7) + (4 .* (x .^
3) .* y_sub) - 5;
    dy = (3 .* (x .^ 2)) - (2 .* y_sub) - (2 .* x .* dy_sub) + (7
.* (y_sub .^ 6) .* dy_sub) + (12 .* (x .^ 2) .* y_sub) + (4 .*
(x .^ 3) .* dy_sub);

end

%
=====
(2) APPROXIMATING ROOT IN script_03.m FILE
%
=====
```

```

tolerance = 10 ^ (-8); % tolerance chosen to be sufficiently
close to
iterations = 50; % chosen arbitrarily to get approximation
x_init = 1; % initial guess specified by problem

% run newtons method with given method
[x_approx, y_approx, iterations_needed, success] = ...
    newtonsMethod(tolerance, iterations, x_init, @(x)
function_03(x));

% print success if we've found a zero within iterations and
tolerance
if success;
    disp( ...
        "NEWTON: Approximate root found successfully in " ...
        + iterations_needed ...
        + " iterations: x = " ...
        + x_approx(end) ...
        );
else;
    disp( ...
        "NEWTON: Approximate root was NOT found within tolerance
in " ...
        + iterations + " iterations." ...
        );
end;

% get exact values of function to find root of
h = 0.0001; % higher accuracy chosen here for better graph,
changes highly on h
start_pt_more = min(x_approx); end_pt_more = max(x_approx); %
initialization values full range
% get the larger range graph
x_exact_more = [start_pt_more:h:end_pt_more]; % calculate exact
for these vals of x
y_exact_more = zeros(size(x_exact_more)); % preallocate memory

counter = 1;
for x = x_exact_more;
    [y, dy] = function_03(x);
    y_exact_more(counter) = y;
end;

```

```

        counter = counter + 1;
end;

% plot the exact graph versus the approximation points
figure(1);

plot(x_exact_more, y_exact_more, "--c", ...
     x_approx, y_approx, "oy", ...
     x_approx(1), y_approx(1), "og", ...
     x_approx(end), y_approx(end), "or", ...
     LineWidth=1.5);
hold on;
legend( ...
    'Exact Values of f(x)', ...
    "Approximations to the Root with Newton's Method", ...
    "Initial Guess, x = " + x_init, ...
    "Final Approximation" ...
);
title("Exact f(x) vs Approximations to the Root using Newton's Method");
xlabel("Domain, [" + start_pt_more + ", " + end_pt_more + "]"
with h = " + h);
ylabel('Range, output of f(x) and Approximations');
ylim([min(y_approx), max(y_approx)]); % for some semblance of
sanity
xlim([min(x_approx), max(x_approx)]);
grid on;

% get the lesser graph for more focused viewing
start_pt_less = 0; end_pt_less = 2; % initialization values less
range
x_exact_less = [start_pt_less:h:end_pt_less]; % calculate exact
for these vals of x
y_exact_less = zeros(size(x_exact_less)); % preallocate memory

counter = 1;
for x = x_exact_less;
    [y, dy] = function_03(x);
    y_exact_less(counter) = y;
    counter = counter + 1;
end;

```



```

% plot the exact graph versus the approximation points
figure(2);
plot(x_exact_less, y_exact_less, "--c", ...
     x_approx, y_approx, "oy", ...
     x_approx(1), y_approx(1), "og", ...
     x_approx(end), y_approx(end), "or", ...
     LineWidth=1.5);
hold on;
legend( ...
    'Exact Values of f(x)', ...
    "Approximations to the Root with Newton's Method", ...
    "Initial Guess, x = " + x_init, ...
    "Final Approximation" ...
);
title("Exact f(x) vs Approximations to the Root using Newton's
Method");
xlabel("Domain, [" + start_pt_less + ", " + end_pt_less + "]
with h = " + h);
ylabel('Range, output of f(x) and Approximations');
ylim([-10, 10]); % for some semblance of sanity
xlim([start_pt_less, end_pt_less]);
grid on;

%
=====
END CODE FOR COMP PROB 3
%
=====

```

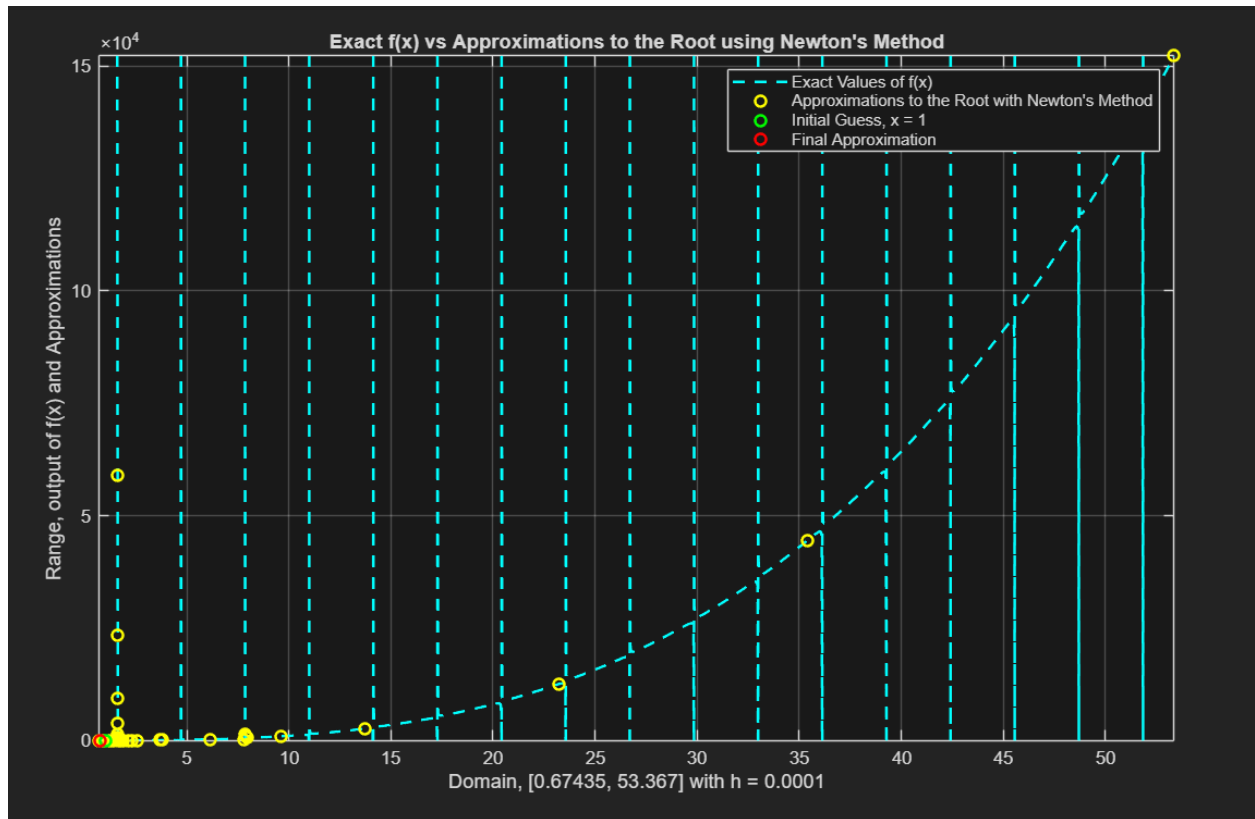
When running the main script, we end up taking 46 iterations to find a root, but we do, finally, at $x = 0.769109$.

```

NEWTON: Approximate root found successfully in 46 iterations: x = 0.76911

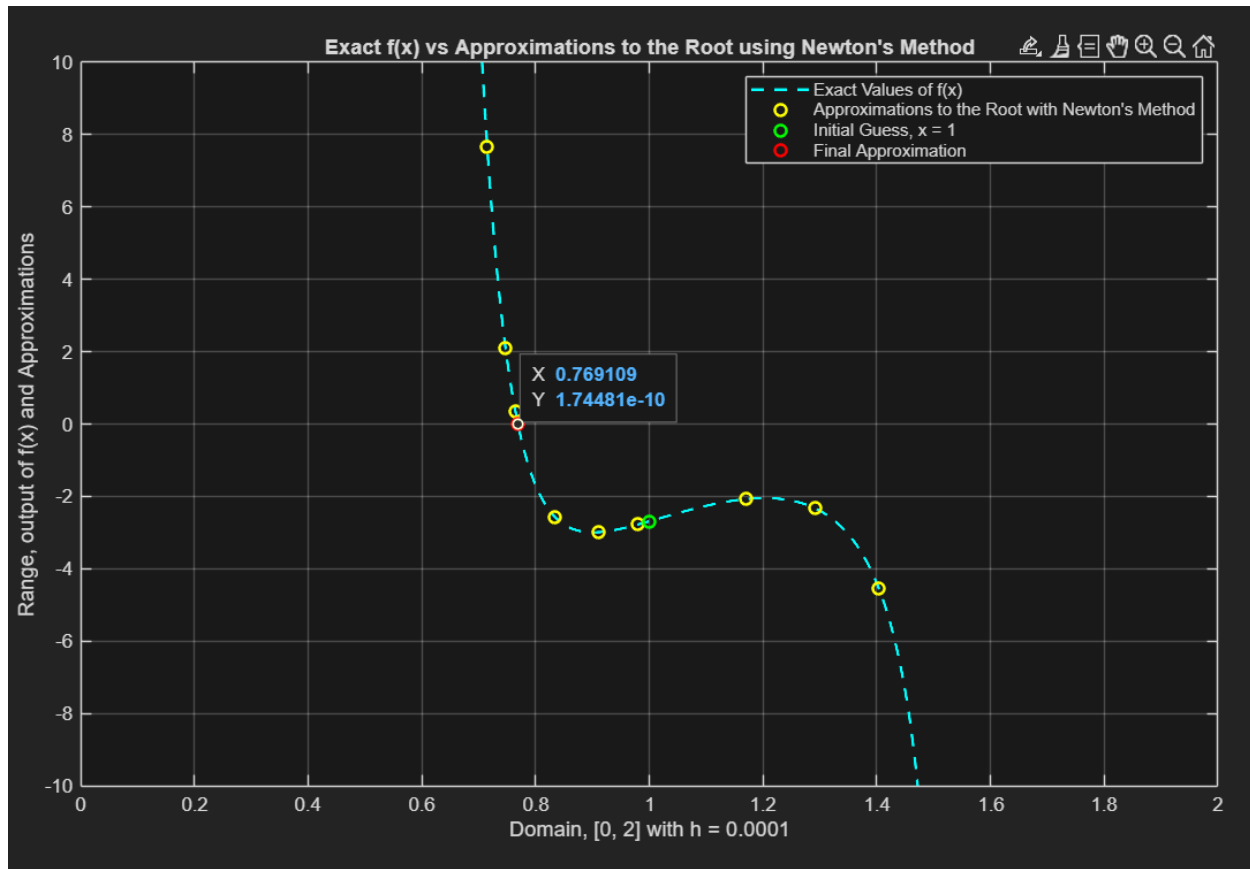
```

The first plot depicted is restricted to the domain of *[lowest x approximation, highest x approximation]*, and the range is similarly restricted to *[lowest y approximation, highest y approximation]*. This was done due to the graph being generated without restrictions being completely incoherent.



We can see a ton of repetitive, yet somewhat chaotic, asymptotic behavior in the function above. For the approximations, we can see some particularly volatile behavior, producing some approximations that are incredibly far from the root found (note the peak $x = 53.367$ at the very top of the interval). However, the method still remarkably converges to the root closest to $x = 1$ given enough iterations.

We plot another figure below that restricts the domain to see closer into the cluster of approximations produced around $x = 1$, cutting out the extraneous approximations produced.



Despite our initial guess being remarkably close to our root at $x = 0.769109$, it took a whopping 46 iterations of Newton's to find it. The function and derivative are incredibly complex to handle, and although the method worked, the convergence took many operations. Let's compare that to using the Bisection Method on $x_{start} = 0.5$ and $x_{end} = 1$ to see how it compares on a similar scale.

When running the below added script, we end up iterating only 30 times to find the same root, and we avoid the unnecessary computation for the gnarly derivative we had to get by hand and code for Newton's.

```
%
=====
EXTRA CODE IN script_03.m FILE, LINES 109 - END
%
=====

x_start = 0.5; % lower bound on interval to find root in
x_end = 1; % upper bound on interval to find root in
```

```

[x_mids, y_mids, iterations_needed, success] = ...
    bisectionMethod(tolerance, iterations, x_start, x_end, @(x)
function_03(x));

% print success if we've found a zero within iterations and
tolerance
if success;
    disp( ...
        "BISECTION: Approximate root found successfully in " ...
        + iterations_needed ...
        + " iterations: x = " ...
        + x_mids(end) ...
        );
    % disp("root found: x = " + x_mids(end));
else;
    disp( ...
        "BISECTION: Approximate root was NOT found within
tolerance in " ...
        + iterations + " iterations." ...
        );
end;

% plot the exact graph versus the approximation points
(BISECTION)
figure(3);
plot(x_exact_more, y_exact_more, "--c", ...
    x_mids, y_mids, "oy", ...
    x_start, function_03(x_start), "og", ...
    x_end, function_03(x_end), "ob", ...
    x_mids(end), y_mids(end), "or", ...
    LineWidth=1.5);
hold on;
legend( ...
    'Exact Values of f(x)', ...
    '(x, y) Approx Midpoints', ...
    'Initial Starting Interval Value', ...
    'Initial Ending Interval Value', ...
    'Final Approximation' ...
    );
title("Exact f(x) vs Approximations to the Root using Bisection
Method");

```

```

xlabel("Domain, [" + start_pt_more + ", " + end_pt_more + "]
with h = " + h);
ylabel('Range, output of f(x) and Approximations');
ylim([min(y_approx), max(y_approx)]); % for some semblance of
sanity
xlim([min(x_approx), max(x_approx)]);
grid on;

% plot the exact graph versus the approximation points
figure(4);
plot(x_exact_less, y_exact_less, "--c", ...
     x_mids, y_mids, "oy", ...
     x_start, function_03(x_start), "og", ...
     x_end, function_03(x_end), "ob", ...
     x_mids(end), y_mids(end), "or", ...
     LineWidth=1.5);
hold on;
legend( ...
    'Exact Values of f(x)', ...
    '(x, y) Approx Midpoints', ...
    'Initial Starting Interval Value', ...
    'Initial Ending Interval Value', ...
    'Final Approximation' ...
);
title("Exact f(x) vs Approximations to the Root using Bisection
Method");
xlabel("Domain, [" + start_pt_less + ", " + end_pt_less + "]
with h = " + h);
ylabel('Range, output of f(x) and Approximations');
ylim([-10, 10]); % for some semblance of sanity
xlim([start_pt_less, end_pt_less]);
grid on;

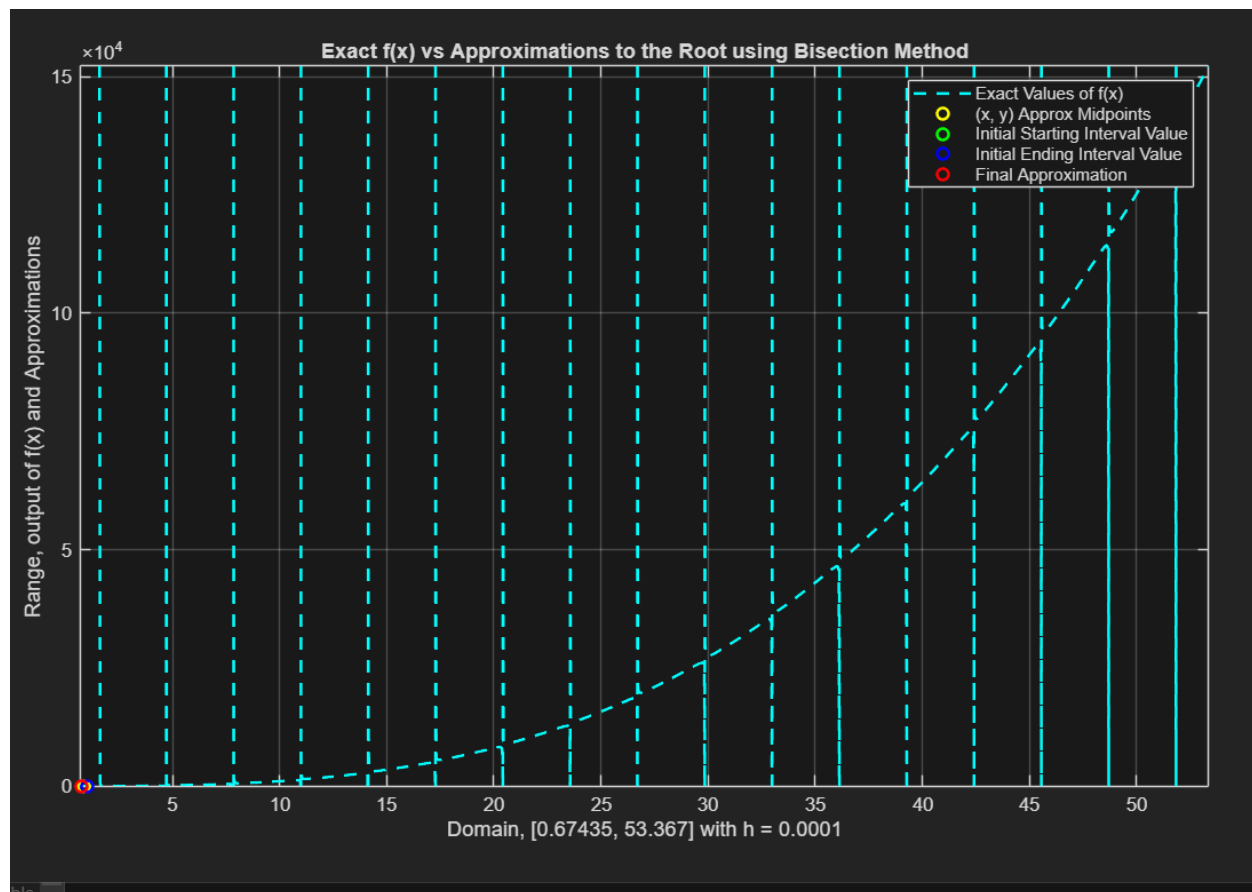
%
=====
END CODE FOR COMP PROB 3
%
=====

```

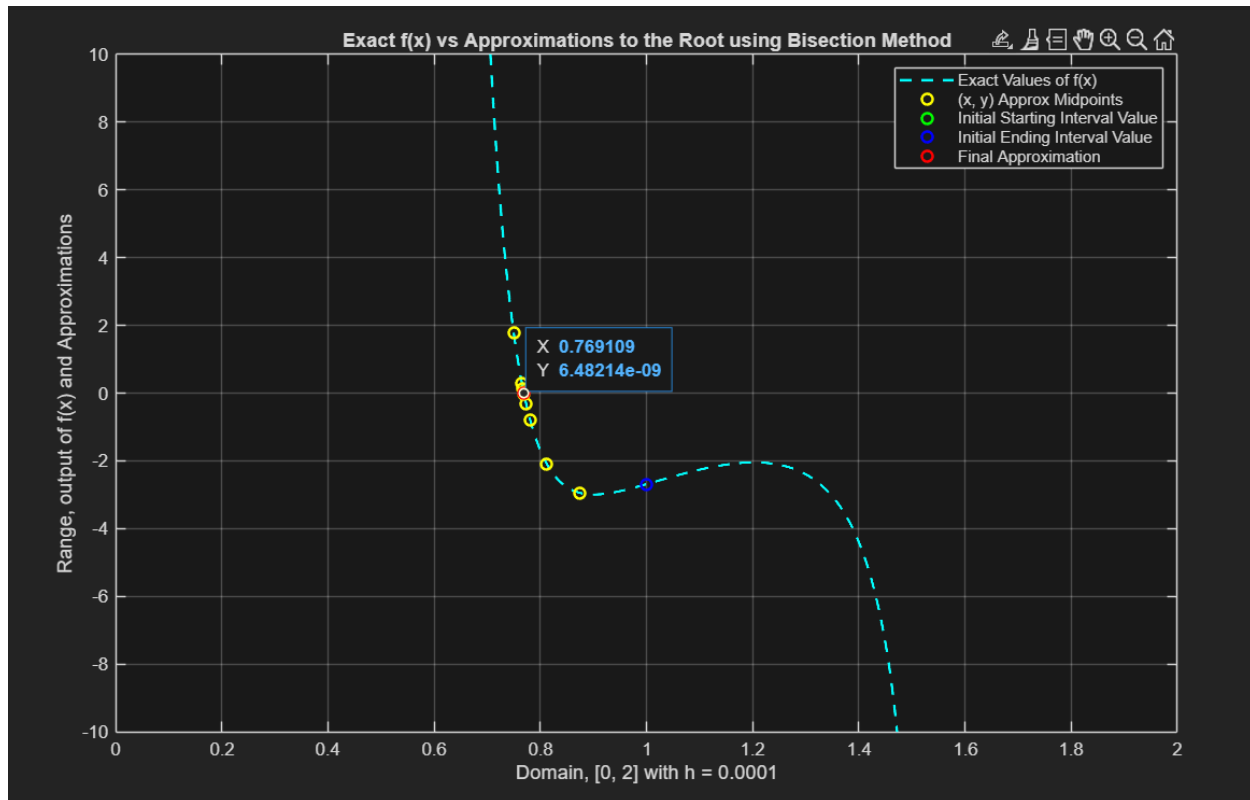
And, as said, on run, we see a notable drop in iterations needed to produce the root:

```
NEWTON: Approximate root found successfully in 46 iterations: x = 0.76911
BISECTION: Approximate root found successfully in 30 iterations: x = 0.76911
```

As seen in the plotting below, we avoid all of those extraneous values the Newton's Method produced. The plot below has the same domain and range as the first plotting of Newton's to highlight the difference in the approximation values.



As we can see, the approximations stay much closer to the actual root. Further, on our zoomed in view below, we can see the mid-point approximations converge.



One observation to note is we see the same effect that occurred in comparisons in Compute Problem 2: Newton's Method yielded $y = 1.74481 \cdot (10^{-10})$ for its final within-tolerance approximation to $y = 0$, whereas Bisection was an order less tight at $y = 6.48214 \cdot (10^{-9})$. However, by picking our interval well with Bisection, we were able to find the same approximation with far less overhead both in iterations (30 vs. 46) and computation (no complicated $f'(x)$ needed).

One limitation to note is that Bisection still does rely on a good interval choice— $x_{start} = 0$ and $x_{end} = 1$ will fail to converge entirely because the function is undefined at $x = 0$ ($\sin(0) + 3(x^2) = 0$, and dividing by 0 is illegal, of course!) However, as long as we are aware of potential start and endpoint conflicts, Bisection Method may actually be a better choice for this function overall.

CONCLUSION

To wrap up all explorations through these problems, we saw some interesting tradeoffs between different methods to approximate the root of a function, namely Newton's Method and the Bisection Method.

Newton's Method ended up being generally a highly accurate method to produce an approximation in the pertinent cases tested in this report. Typically, with a good initial guess, the method can also produce a root within very few iterations. However, some considerations need to be made before blindly using the method. A poor initial guess can lead the method astray, and at times, cause the method to completely fail to find anything. Further, there is the extra work of finding, coding, and computing the derivative of our function, which can be worthwhile, or can just introduce unnecessary complexity when handling a complex function.

The Bisection Method, to contrast, is much simpler in its approach, behaving more as a brute force style algorithm. Because of this, it can take far longer to find a root approximation, and therefore can be far more costly than Newton. It is nevertheless up to the programmer to decide on the tradeoffs. We did explore that, in noisier functions, this can actually be a better, less computationally intensive method, with the added benefit of no derivative calculation needed. It is still up to us to choose our beginning interval wisely, akin to the initial guess of Newton's, but there is a benefit to consider in certain situations.

Overall, the two methods accomplish the same goal. Regardless, it is up to the programmer's responsibility to pick the more appropriate approach depending on the situation, and to move forward wisely.